The Throwable Hierarchy

10 min read · Updated 28 Jul 2026 · Topic 26 of 65

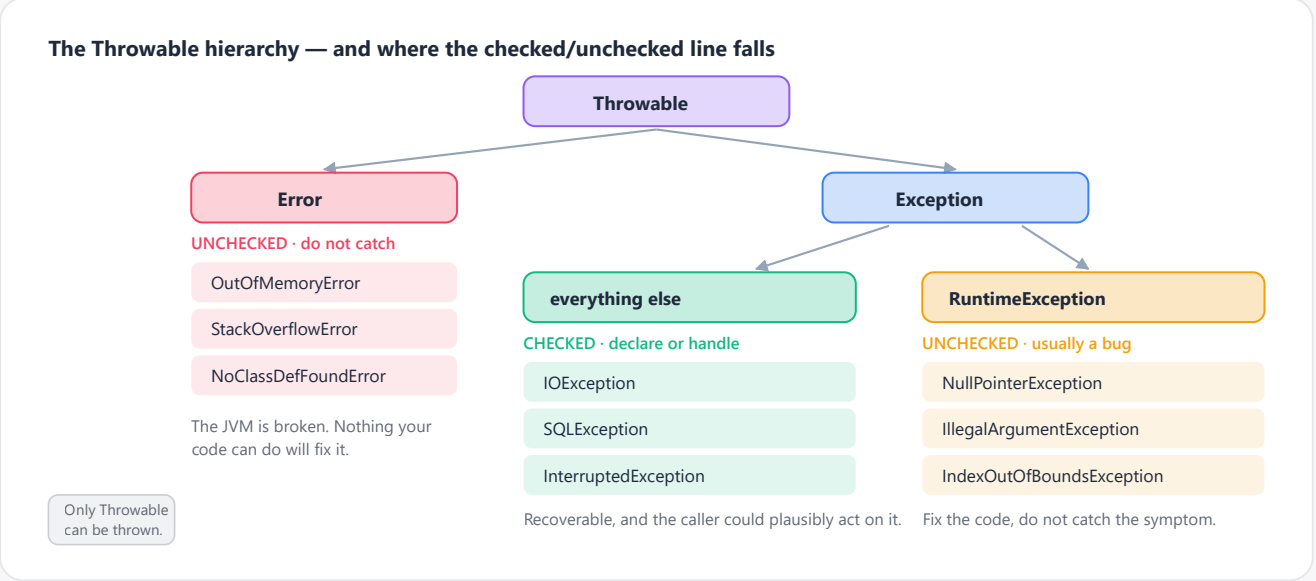
Bookmark · Mark done

Why it matters

Almost every exception question reduces to "where does this sit in the hierarchy, and what does that imply?". Knowing that `Error` and `RuntimeException` are unchecked while everything else under `Exception` is checked tells you what the compiler will demand, what you should catch, and — most importantly — what you should never catch. It also explains why `catch (Exception e)` is usually wrong and `catch (Throwable t)` is almost always a bug.

The concept, simply

Only objects descending from `java.lang.Throwable` can be thrown. Below it, two branches with completely different intentions:



The checked/unchecked line is not the Error/Exception line. Unchecked means Error OR RuntimeException; everything else under Exception is checked.

- `Error` — the JVM or the environment is broken. `OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`. Unchecked, and you should not catch them: there is nothing sensible your code can do.
- `RuntimeException` — a programming mistake. `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException`. Unchecked, because the fix is to correct the code, not to handle the symptom.
- **Everything else under** `Exception` — recoverable conditions outside your control. `IOException`, `SQLException`, `InterruptedException`. Checked: the compiler forces you to declare or handle them.

How it works

The **handle-or-declare rule** applies only to checked exceptions: a method that can throw one must either catch it or list it in a `throws` clause. Unchecked exceptions may propagate freely with no declaration.

When something is thrown, the JVM unwinds the call stack looking for the first `catch` whose type is compatible. Catch blocks are tried **in source order**, which is why a more general type must come after a more specific one — the reverse does not compile ("alternative has already been caught").

Every `Throwable` carries a **stack trace** captured at construction — not at throw. That distinction matters: creating an exception and throwing it later records where it was *created*. It is also why exception creation is expensive, since capturing a trace walks the stack.

Chaining is the mechanism that keeps a root cause alive across abstraction layers. `new ServiceException("could not load user", e)` stores `e` as the cause, and the printed trace shows both with a "Caused by:" section. Losing the cause is one of the most common and most costly logging mistakes.

Code examples

Ordering and multi-catch:

JAVA

Copy

```
} catch (FileNotFoundException e) {      // ✓ most specific first
    log.warn("missing file {}", file, e);
} catch (IOException e) {                // ✓ broader afterwards
    log.error("I/O failure", e);
}

// ✗ Reversed, this does not compile:
// catch (IOException e) { }
// catch (FileNotFoundException e) { }   → "has already been caught"

// Multi-catch (Java 7+) when the handling is identical. The variable is
// implicitly final, and its static type is the common supertype.
try {
    parseAndSend(payload);
} catch (ParseException | IllegalStateException e) {
    throw new ProcessingException("bad payload", e);
}
```

Chaining, and the mistake that destroys it:

JAVA Copy

```
// ✗ The original cause is gone. The trace points at your wrapper and the
//   real failure — a connection timeout, say — is unrecoverable.
try {
    repository.load(id);
} catch (SQLException e) {
    throw new ServiceException("could not load user " + id);
}

// ✓ Pass the cause. The trace shows "Caused by: java.sql.SQLException: ..
try {
    repository.load(id);
} catch (SQLException e) {
    throw new ServiceException("could not load user " + id, e);
}

// ✗ Also destroys it — logging only the message, not the exception:
```

What not to catch:

JAVA

Copy

```
// ✗ Catches Errors too — including OutOfMemoryError and StackOverflowErro
//   The JVM is in an unknown state and you have just decided to continue.
try { work(); } catch (Throwable t) { log.error("oops", t); }

// ✗ Catches every RuntimeException, so genuine bugs are swallowed and
//   the application limps on in a broken state.
try { work(); } catch (Exception e) { /* ignored */ }

// ✓ Catch what you can actually handle, and let the rest propagate.
try { work(); } catch (IOException e) { retryLater(); }

// The one legitimate broad catch: a top-level boundary that must not die
// a request handler or a worker loop — where you log and return an error.
try {
    handle(request);
} catch (Exception e) {                 // Exception, NOT Throwable
    log.error("request {} failed", request.id(), e);
    return Response.serverError();
}
```

The one `Error` people do catch, and why it is still a bad idea:

JAVA

Copy

```
// StackOverflowError is "recoverable" in the sense that the stack unwinds
// but you cannot know how much work was half-completed. Treat it as a bug
// the recursion, not a condition to handle.
//
// InterruptedException, by contrast, is checked and MUST be handled prope
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();   // ✓ restore the interrupt flag
```

```
    throw new CancellationException("interrupted while waiting");
}
// Swallowing InterruptedException silently breaks every cancellation
// mechanism above you. Chapter 8 covers why.
```

Under the hood

Exceptions are expensive, and the cost is the stack trace. `fillInStackTrace()` walks the entire call stack at construction. Throwing and catching itself is cheap; capturing the trace is not. This is why exceptions must never be used for control flow in a hot path. If you genuinely need a lightweight signalling exception — a common trick in parsers and stream short-circuits — you can override `fillInStackTrace()` to do nothing, or use the four-argument `Throwable` constructor with `writableStackTrace = false`.

The stack trace is captured at construction, not at throw. A pre-allocated or cached exception rethrown from many places will show the original creation site every time — deeply confusing during debugging, and a real hazard with the JIT optimisation below.

Stack traces can disappear entirely. HotSpot has an optimisation where a frequently-thrown implicit exception (an NPE or array-index exception from the same site, thrown thousands of times) is recompiled to throw a pre-allocated instance with an *empty* stack trace. The symptom is a production log full of `java.lang.NullPointerException` with no trace at all. The fix is `-XX:-OmitStackTraceInFastThrow`, and knowing this is a strong senior signal because the alternative explanation — "the logging framework is broken" — sends people down a long wrong path.

Helpful NullPointerExceptions (Java 14+, on by default since 15) are worth naming as the modern counterweight: instead of "NullPointerException" the JVM now says *"Cannot invoke String.length() because the return value of Map.get(Object) is null"*, naming the exact expression. It removes most of the guesswork from the commonest exception in Java.

Common mistakes

- **Catching `Throwable`.** That includes `Error`. The JVM is broken; continuing is worse than failing.

- **Empty catch blocks.** The single worst thing in Java code — the failure happened and nobody will ever know.
- **Losing the cause.** Always pass the original exception into the wrapper's constructor.
- `log.error(e.getMessage())`. Drops the stack trace. Pass the exception itself.
- **Log and rethrow.** The same failure gets logged at every layer. Log once, at the boundary that handles it.
- **Exceptions for control flow.** Constructing them walks the stack — expensive, and it misuses the mechanism.
- `e.printStackTrace()` **in production.** Goes to stderr, unstructured, invisible to your log aggregation.
- **Swallowing** `InterruptedException`**.** Restore the flag or rethrow; otherwise cancellation silently stops working.

Interview lens

Q: "Describe the exception hierarchy."
A: `Throwable` at the root, with two branches. `Error` means the JVM or environment has failed — `OutOfMemoryError`, `StackOverflowError` — and should not be caught. `Exception` covers application-level problems and splits again: `RuntimeException` and its subclasses are unchecked and usually indicate bugs, while everything else under `Exception` is checked and must be declared or handled. So "unchecked" means `Error` or `RuntimeException`, which is not the same as the `Error`/`Exception` split.

Q: "Should you ever catch `Error`**?"**
A: Essentially never. An `Error` means something the application cannot fix — memory exhausted, a class missing, the stack blown. Catching it lets a process continue in an unknown state, which usually converts a clean crash into corrupted data. The only defensible case is a top-level handler that logs and then terminates deliberately.

A: Because it catches every `RuntimeException` too, so real bugs — an NPE, an illegal argument — get treated as expected conditions and the application continues in a broken state. I catch the specific types I can actually handle. The exception is a genuine boundary, like a request handler or a worker loop, where the process must not die; there I catch `Exception`, log with the exception object, and return an error — but never `Throwable`.

Q: "What is exception chaining and why does it matter?"

A: Passing the original exception as the `cause` when you wrap it, so the printed trace includes a "Caused by:" section with the root failure. Without it you get a wrapper exception pointing at your own code and no indication that the real problem was, say, a connection timeout. It is the difference between a five-minute diagnosis and an afternoon.

Q: "Are exceptions expensive?"

A: Constructing them is, because `fillInStackTrace` walks the whole call stack. Throwing and catching is comparatively cheap. So they are fine for exceptional conditions and wrong for control flow in a hot loop. If you truly need a fast signalling exception you can suppress the trace by overriding `fillInStackTrace` or using the constructor with `writableStackTrace = false`.

Q: "You see NullPointerExceptions in production with no stack trace. Why?"

A: Almost certainly HotSpot's fast-throw optimisation. After the same implicit exception is thrown from one site many times, the JIT recompiles it to throw a pre-allocated instance with an empty trace. Disable it with `-XX:-OmitStackTraceInFastThrow` to get traces back. It is worth checking before assuming the logging configuration is at fault.

A: Never swallow it. Either propagate it, or catch it and immediately call `Thread.currentThread().interrupt()` to restore the flag the catch cleared. If you do neither, code higher up that relies on the interrupt to stop work will never see it, and cancellation and shutdown quietly stop functioning.

Cheat-card

- `Throwable` → `Error` (do not catch) + `Exception` → `RuntimeException` (unchecked) and the rest (checked).
- **Unchecked =** `Error` **or** `RuntimeException`. Everything else is checked.
- Handle-or-declare applies to **checked only**.
- Catch blocks are tried in **source order** — specific before general, or it will not compile.
- Always **chain the cause**; always `log.error("msg", e)`, never `e.getMessage()`.
- Never catch `Throwable`. Never write an empty catch. Never log-and-rethrow.
- Cost is `fillInStackTrace` — captured at **construction**, not at throw.
- Missing traces in production → `-XX:-OmitStackTraceInFastThrow`.
- `InterruptedException` → rethrow or **restore the interrupt flag**.

QUICK CHECK — no marks, just for you.

1. Which statement about the exception hierarchy is correct?

- ○ Unchecked means Error OR RuntimeException; everything else under Exception is checked
- ○ Unchecked means anything under Error; everything under Exception is checked
- ○ All exceptions are checked; only Errors are unchecked
- ○ RuntimeException is checked because it extends Exception

2. Why is catching Throwable almost always a bug?

- ◯ Because it is slower than catching Exception
- ◯ Because it includes Error — OutOfMemoryError, StackOverflowError — so you continue running with the JVM in an unknown state
- ◯ Because Throwable is abstract
- ◯ Because the compiler forbids it

Check answer

3. Your production logs show NullPointerException with no stack trace at all. What is the most likely cause?

- ◯ The logging framework is misconfigured
- ◯ The exception was constructed with a null message
- ◯ HotSpot's fast-throw optimisation: after the same implicit exception is thrown many times from one site, the JIT recompiles it to throw a pre-allocated instance with an empty trace
- ◯ The stack was too deep to capture

Check answer

Java Interview Kit

LEARN MORE

ARTICLES & DOCS

Throwable — API docs
docs.oracle.com

Exceptions — The Java Tutorials
docs.oracle.com

VIDEOS



Master Exceptions in Java: Try, Catch, Finally, Throw, Throws, try-with-resources & Custom Exception
Engineering Digest

Hand-picked references for this lesson.